

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class EmotionNet(nn.Module):
    def __init__(self, input_dim=32,
hidden_dim=64, n_emotions=6):
        super().__init__()
        self.backbone = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU()
        )

        self.valence_head =
nn.Linear(hidden_dim, 1)
        self.arousal_head =
nn.Linear(hidden_dim, 1)
        self.emotion_head =
nn.Linear(hidden_dim + 2, n_emotions)
```

```
def forward(self, x):
    h = self.backbone(x)
    valence =
torch.tanh(self.valence_head(h))
    arousal =
torch.tanh(self.arousal_head(h))
    z = torch.cat([h, valence, arousal], dim=-1)
    logits = self.emotion_head(z)
    probs = F.softmax(logits, dim=-1)
    return {
        "valence": valence,
        "arousal": arousal,
        "logits": logits,
        "probs": probs
    }
```

```
if __name__ == "__main__":
    model = EmotionNet(input_dim=32,
hidden_dim=64, n_emotions=6)
    x = torch.randn(4, 32)
    out = model(x)
```

```
print("Valence:", out["valence"].shape)
print("Arousal:", out["arousal"].shape)
print("Emotion probabilities:", out["probs"])
```